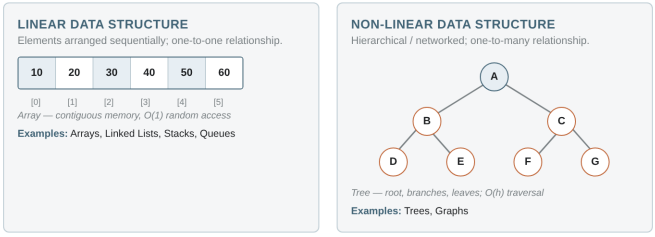
DSA Lecture 1 — Cheatsheet

Data Structure & Algorithm · Introduction, Complexity & Asymptotic Notation

1. Introduction to Data Structures

Data Structure — the logical or mathematical model of a particular organization of data. Choice of structure depends on two considerations: (i) it must be rich enough to mirror real-world relationships of the data, and (ii) it must be simple enough to process data effectively when needed.



Types

Linear — elements arranged sequentially; one-to-one relationship.
Examples: Arrays, Linked Lists, Stacks, Queues.

Non-linear — hierarchical or networked; one-to-many relationship.
Examples: Trees, Graphs.

2. Data Structure Operations

Operation	Description
Traverse	Access each record exactly once for processing.
Search	Find the location of a record with a given key value.
Insert	Add a new record to the structure.
Delete	Remove a record from the structure.
Sort	Arrange records in some logical order.
Merge	Combine two sorted files into a single sorted file.

3. Algorithms

Algorithm — a well-defined set of instructions used to solve a particular problem. Below is a worked example: finding the location of the largest element of an array *Data* of size *N*.

EXAMPLE · Find max element — pseudocode

```
Largest-Item (Data, N, Loc)
1.  set k := 1,  Loc := 1,  Max := Data[1]
2.  while k <= N repeat steps 3, 4
3.       if Max < Data[k] then
             set Loc := k  and  Max := Data[k]
4.       set k := k + 1
5.  write: Max and Loc
6.  exit
```

4. Complexity of Algorithms

The **complexity** of an algorithm M is the function *f(n)* giving the running time and/or storage space requirement in terms of the input size *n*. Two types:

Time Complexity — how fast the algorithm runs as *n* grows.
Space Complexity — how much memory the algorithm uses as *n* grows.

Time–space tradeoff: by increasing storage we may reduce processing time, or vice versa. Choosing a data structure often involves this tradeoff.

5. Analyzing Algorithms

Running time = the number of primitive operations (steps) executed before termination. Primitive operations include arithmetic (+, −, ×), data movement, control flow, decision making (if, while), and comparison. Running time **grows with input size n** — e.g. array size, polynomial degree, # of matrix elements, # of bits, vertices/edges in a graph.

Worked example — MIN(a[1..n])

```
Alg.:  MIN(a[1], ..., a[n])
    m <- a[1];
    for i <- 2 to n
        if a[i] < m  then  m <- a[i];
```

Counting primitive operations: **1** (init) + **n** (for-loop) + **(n−1)** (if) + **(n−1)** (assignment) = **T(n) = 3n − 1**. The **order of growth** is the leading term — it expresses the asymptotic behavior.

6. Typical Running Time Functions

Function	Name	Description
`1`	Constant	Instructions run once or a few times.
`log N`	Logarithmic	Problem cut to a constant fraction of its size each step.
`N`	Linear	A small amount of processing on each input element.
`N log N`	Linearithmic	Divide into subproblems, solve, combine.
N^2	Quadratic	All pairs of items (double nested loops).
N^3	Cubic	All triples of items (triple nested loops).
N^k	Polynomial	k nested loops over the input.
2^N	Exponential	Few are practical — exhaustive enumeration.

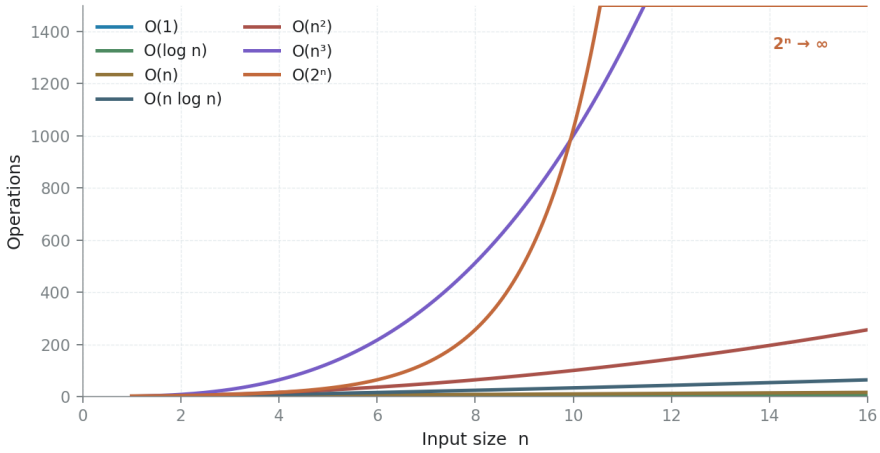


Figure 1. Growth of common complexity functions. Note how 2^n explodes while log n stays nearly flat.

7. Best / Worst / Average Case

Case	Definition
Worst case	Maximum # of steps on any instance of size n. (Usually reported.)
Best case	Minimum # of steps on any instance of size n. (Often bogus / misleading.)
Average case	Average # of steps over all instances of size n — requires input distribution.

String-search example. Searching for character *'a'* in a string of length *n*: **O(1)** best case (string starts with 'a', e.g. "alibaba"); **O(n)** worst case (no 'a' present, e.g. "nitin"); **O(n)** average case. This shows input size alone is **not** enough to determine time complexity unambiguously — the actual input matters too.

Why worst case? Easier to compute, usually close to the actual running time, and provides a *guarantee* that the algorithm won't take longer, no matter the input. Strategy: find a function that, for large n, is an **upper bound** on the actual step count.

8. Asymptotic Notation

Asymptotic analysis compares and classifies functions while **ignoring constant factors** and **small inputs**. Three sets are used — Big-O (upper bound), Big-Ω (lower bound), and Big-Θ (tight bound).

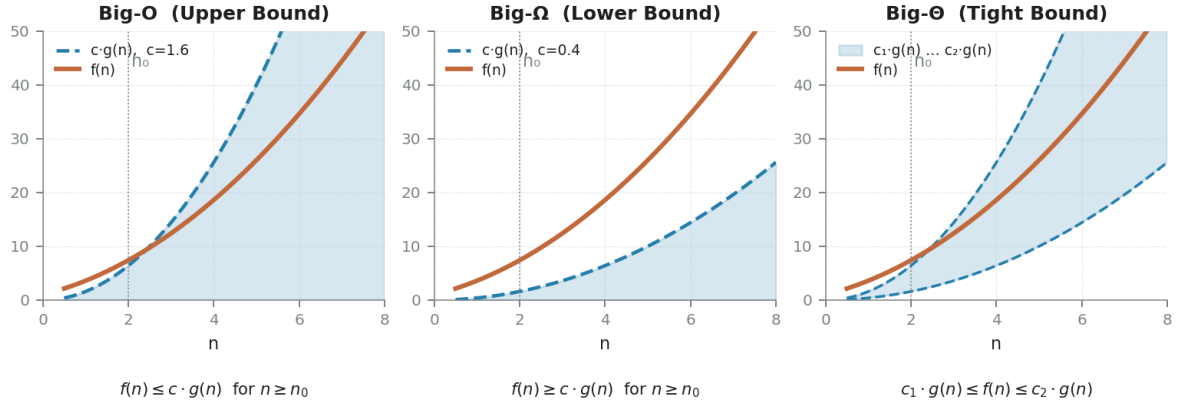


Figure 2. Visualization of Big-O, Big-Ω, and Big-Θ bounds. f(n) is the actual algorithm; c·g(n) is the reference function.

Notation	Definition	Meaning
`Big-O (O)`	Asymptotic **upper** bound. $\exists$ c, $n_0 > 0$ such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$.	f does not grow faster than g.
`Big-Ω (Ω)`	Asymptotic **lower** bound. $\exists$ c, $n_0 > 0$ such that $0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0$.	f grows at least as fast as g.
`Big-θ (θ)`	Asymptotic **tight** bound. $\exists$ c_1, c_2, $n_0 > 0$ such that $0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$.	f grows exactly like g.

Big-O interpretation

If **$f(n) = O(n^2)$**, then f(n) can exceed n^2 occasionally, but we can choose constants c and n_0 so that for every $n > n_0$: **$f(n) \le c \cdot n^2$**. In words: f(n) does not grow more than a constant factor faster than n^2. **We want the tightest bound** — saying $n^2 + 7n$ is $O(n^2)$ is more interesting than saying it is $O(n^3)$.

Worked examples — picking c and n_0

Claim	Proof (pick constants)
$2n^2 = O(n^3)$	$2n^2 \le c \cdot n^3 \Rightarrow c = 1,\ n_0 = 2$
$n^2 = O(n^2)$	$n^2 \le c \cdot n^2 \Rightarrow c = 1,\ n_0 = 1$
$1000n^2 + 1000n = O(n^2)$	$c = 1001,\ n_0 = 1$
$n = O(n^2)$	$n \le c \cdot n^2 \Rightarrow c = 1,\ n_0 = 1$
$n^2 = \Omega(n)$	$n^2 \ge c \cdot n \Rightarrow c = 1,\ n_0 = 2$
$200n^2 = \theta(n^2)$	$c_1 = 1,\ c_2 = 200,\ n_0 = 1$
$n \ne \theta(n^2)$	$n = O(n^2)$ but n is not $\Omega(n^2)$.

9. Simplifying Rules for Big-O

Rule	Statement
1. Transitivity	If f(n) = O(g(n)) and g(n) = O(h(n)), then f(n) = O(h(n)).
2. Constant factor	If f(n) = O(k·g(n)) for any k > 0, then f(n) = O(g(n)).
3. Sum rule	If $f_1 = O(g_1)$ and $f_2 = O(g_2)$, then $f_1 + f_2 = O(\max(g_1, g_2))$.
4. Product rule	If $f_1 = O(g_1)$ and $f_2 = O(g_2)$, then $f_1 \cdot f_2 = O(g_1 \cdot g_2)$.

10. Code Examples — Time Complexity

```
count = 0;
for (i=0; i<10000; i++)
    count++;
```

Complexity: O(1) Loop runs a constant number of times.

```
count = 0;
for (i=0; i<n; i++)
    count++;
```

Complexity: O(n) Single loop, n iterations.

```
sum = 0;
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        sum += arr[i][j];
```

Complexity: O(n^2) Two nested loops, each n iterations.

```
sum = 0;
for (i=1; i<=n; i=i*2)
    sum += i;
```

Complexity: O(log n) i doubles each step → $\log_2$ n iterations.

```
sum = 0;
for (i=1; i<=n; i=i*2)
    for (j=0; j<n; j++)
        sum += i*j;
```

Complexity: O(n log n) Outer log n × inner n.

```
sum = 0;
for (i=1; i<=n; i=i*2)
    for (j=0; j<i; j++)
        sum += i*j;
```

Complexity: Θ(n) (tight) Loose O(n log n); tight bound is n.

11. Recurrences — Preview

Recurrence — an equation or inequality that describes a function in terms of its value on smaller inputs, plus one or more base cases. Example: **$T(n) = T(n-1) + n$**. Used to analyze recursive algorithms. **Solving methods:** substitution, recursion-tree, master theorem, iteration.

⚠ Common Pitfalls

- **$O \neq \Theta$.** $n = O(n^2)$ is true, but $n \neq \Theta(n^2)$. O is an upper bound, Θ is tight.
- **Best case is bogus.** Don't quote best-case complexity as if it were typical.
- **Constants do not matter** in asymptotic analysis — drop them.
- **Loose vs tight:** always report the tightest bound you can prove.
- **i < i in inner loop** (where outer i = i*2) is NOT n^2 — sum is geometric.

✎ Exam Tips

- **Single loop over n** → O(n). **Two nested** → $O(n^2)$.
- **i = i * 2 (or /2)** → loop runs $\log_2 n$ times.
- **Nested loops multiply:** outer × inner complexity.
- **Sum rule:** sequential blocks → take the max complexity.
- **Always justify** c and n_0 when proving O / Ω / Θ formally.
- **Order ranking to memorize:** $1 < \log n < n < n \log n < n^2 < n^3 < 2^n$

12. Quick Reference — Complexity Comparison

Complexity	Name	Example Algorithms	Verdict
`O(1)`	Constant	Hash table lookup, array index	Excellent
`O(log n)`	Logarithmic	Binary search, balanced BST ops	Excellent
`O(n)`	Linear	Linear scan, single loop	Good
`O(n log n)`	Linearithmic	Merge sort, heap sort, FFT	Good
$O(n^2)$	Quadratic	Bubble / insertion / selection sort	Fair
$O(n^3)$	Cubic	Matrix multiplication (naive)	Poor
2^n	Exponential	Tower of Hanoi, subset enumeration	Intractable